



Data Structures and Algorithms -3 (25CS2103E)

Dictionary-Based Misspelled Word Detection System

Team No : 16

Section : 7

Hansika Reddy Gunapati :2520030237

Adoori Teja Hasini Reddy : 2520030048

Anthony Nissy : 2520030064

Under the Esteemed Guidance of

Dr. S. Madhavi

Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope

Abstract

- The Dictionary-Based Misspelled Word Detection System is a text-processing application that identifies misspelled words in documents, log files, and user-generated text. It compares input text with a predefined dictionary using efficient techniques such as Trie, Aho–Corasick, and Suffix Arrays for faster searching than naïve scanning.
- Words not found in the dictionary are flagged as possible errors. Levenshtein Edit Distance, Soundex, and Metaphone are then used to generate and rank correction suggestions. The system produces a structured error report and supports Unicode and multilingual text.
- The project demonstrates the practical use of data structures and algorithms in text processing and can be evaluated using accuracy, lookup time, and scalability.

Introduction

- Spelling mistakes are common in documents, user input, logs, and digital text. They can reduce readability, cause misunderstanding, and affect text-processing systems. Therefore, an efficient method for detecting and correcting spelling errors is important.
- Traditional spellcheckers often compare each input word individually with dictionary words, which can become inefficient for large datasets. This project proposes a Dictionary-Based Misspelled Word Detection System using efficient data structures and algorithms.
- The system uses Trie and Aho–Corasick for fast string matching, while Suffix Arrays support partial or substring searching. For detected misspellings, Edit Distance and phonetic matching generate correction suggestions. The results are compiled into a structured report, with support for Unicode and multilingual dictionaries.

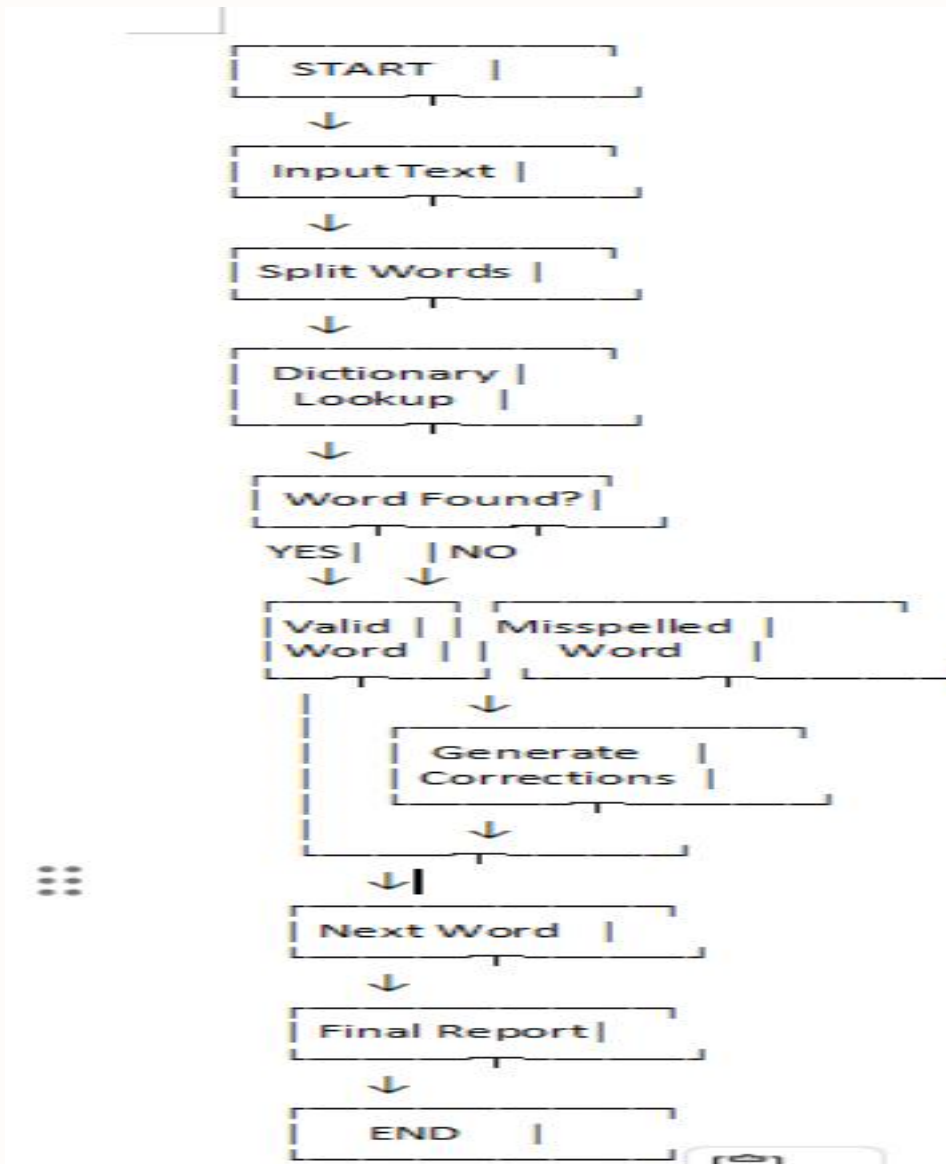
Literature Survey

- Spellchecking traditionally uses dictionary-based methods to identify valid and invalid words.
- Naïve searching can be slow for large dictionaries, while efficient data structures improve lookup performance.
- Trie enables fast word and prefix searching.
- Aho–Corasick supports efficient multi-pattern matching while scanning text.
- Suffix Arrays are useful for substring and partial-pattern searches.
- Levenshtein Edit Distance ranks corrections using insertions, deletions, and substitutions.
- Soundex/Metaphone improve matching of words with similar pronunciation.
- Combining dictionary lookup with correction techniques provides a more complete spellchecking system.
- The project also supports Unicode, multilingual text, and performance comparison with naïve scanning.

Problem Statement and Objectives

- Spelling errors are common in digital text, and traditional dictionary scanning can become slow when processing large dictionaries and text datasets. There is a need for an efficient system that can quickly detect misspelled words, generate accurate correction suggestions, and handle large, Unicode, and multilingual dataset
- Develop a dictionary-based misspelled-word detection system with efficient word storage and searching.
- Implement Trie-based lookup and explore Aho–Corasick for multi-pattern matching.
- Use Suffix Arrays for partial and substring searching.
- Detect words that are absent from the predefined dictionary.
- Generate and rank correction suggestions using Levenshtein Edit Distance.
- Generate a structured error and correction report.
- Support Unicode and multilingual dictionaries.

Flow Chart



Results

- Each word is checked against a predefined dictionary, and unknown words are flagged as possible misspellings. Correction candidates are generated using Edit Distance and phonetic matching , then ranked by similarity.
- A structured report contains the misspelled word, suggested correction, method, and score.
- Examples: recieve → receive, seperate → separate, fone → phone, definately → definitely, acheive → achieve, thier → their.
- The system can be evaluated using detection accuracy, Top-1/Top-3 correction accuracy, lookup latency, and scalability . Illustrative performance values should be replaced with actual measured results when available.

Hardware and Software requirements

Hardware Requirements

- Laptop or desktop computer.
- Minimum 4 GB RAM with a dual-core processor or higher.
- Adequate storage for programs, dictionaries, and datasets.
- Keyboard and mouse.
- Internet connection for downloading datasets or libraries.

Software Requirements

- OS: Windows, Linux, or macOS.
- Language: Python or Java.
- Data Structure: Trie.
- String Matching: Aho–Corasick.
- Searching: Suffix Array.
- Correction: Levenshtein Edit Distance.
- Phonetic Matching: Soundex / Metaphone.
- Dictionary: Curated word lists.
- Input: Text, documents, logs, or user-generated text.
- Testing: CSV/text datasets with UTF-8 encoding.
- Output: Console or structured reports.
- Benchmarking: Performance profiling of search methods.

Conclusion

- The project provides a systematic approach for detecting spelling errors.
- It uses a predefined dictionary for validating words.
- Trie provides efficient prefix-based word lookup.
- Aho–Corasick supports efficient multi-pattern matching.
- Suffix Arrays provide support for partial or substring searches.
- Levenshtein Edit Distance helps identify similar valid words.
- Soundex and Metaphone provide phonetic correction support.
- Combining these techniques improves the overall functionality of the spellchecking system.
- The system can process large text datasets.
- It can support Unicode and multilingual dictionaries.
- The system generates a structured report of detected errors.
- Performance can be compared with naïve scanning.
- The project demonstrates the practical application of Data Structures and Algorithms to a real-world text-processing problem.
- The system can be integrated into text analytics and NLP workflows.

Future Scope

- Implement context-aware spelling correction.
- Use n-gram language models to understand the context of words.
- Detect cases where a word is correctly spelled but incorrectly used.
- Expand the system with more multilingual dictionaries.
- Add support for additional Unicode languages.
- Develop real-time spellchecking while users type.
- Integrate the system with word processors.
- Develop a web-based spellchecking interface.
- Improve correction ranking using machine-learning techniques.
- Add domain-specific dictionaries.
- Support technical, medical, academic, and professional vocabulary.
- Improve handling of very large text datasets.
- Integrate the system with existing NLP pipelines.
- Provide better visualization of detected spelling errors.
- Generate detailed performance and correction reports.
- Extend the system for automated document-review applications.
- Improve scalability for large-scale text analytics

Thank you